



2025 Fall Semester Term Project

Reproducing NightReID: A Paper and Code Review of the EDA Framework for Nighttime Person Re-Identification

과목: 딥러닝의 기초와 프로그래밍
스쿨: 한국과학기술연구원 스쿨
담당교수: 강동훈 교수님
전공: AI·로봇 전공
이름: 허정민

1 Abstract

Nighttime person re-identification (Re-ID) is challenging due to low illumination, noise, and strong domain shifts from daytime data. The NightReID benchmark introduces a large-scale nighttime dataset and the EDA (Enhancement–Denoising–Alignment) framework to alleviate these issues. In this project, we review the NightReID paper and official codebase and reproduce the authors’ results. Our reproduction achieves 51.4% mAP and 66.6% Rank-1 under the 528 IDs w/ distractor setting, closely matching the reported performance.

2 Introduction

2.1 Background

Person Re-Identification (Re-ID) aims to retrieve images of the same identity across non-overlapping camera views. While most existing studies have focused on well-lit daytime scenarios, nighttime Re-ID remains relatively underexplored despite its critical importance in 24-hour intelligent surveillance systems.

In the domain of daytime Re-ID, Luo et al. [3] proposed a strong baseline based on ResNet50, introducing various training strategies to significantly enhance performance. Subsequently, the TransReID model, a ViT-based approach proposed by He et al. [2], achieved superior feature extraction by leveraging global attention. They achieved high performance by incorporating local body part features and camera view information through the Jigsaw Patch Module (JPM) and Side Information Embedding (SIE). Furthermore, Luo et al. [4] improved the TransReID architecture by applying self-supervised learning (SSL), enabling training on large-scale unlabeled datasets. These state-of-the-art (SOTA) models have demonstrated superior performance on standard daytime datasets such as Market1501 and MSMT17.

2.2 Motivation

Despite these technological advancements, applying these models directly to nighttime environments remains a significant challenge. Nighttime scenarios present additional complexities compared to daytime environments. The primary issue is image quality degradation caused by low illumination. In poorly lit conditions, sensors fail to capture sufficient light, resulting in the loss of detailed features and increased noise levels. Additionally, artificial lighting (e.g., streetlights, headlights) introduces severe color distortions, which degrade the color cues learned from daytime data.

In this context, We selected the paper "NightReID: A Large-Scale Nighttime Person Re-Identification Benchmark" for my term project because it introduces a large-scale real-world dataset specifically tailored for these challenging nighttime scenarios. The main objective of this project is to analyze the authors’ approach to addressing the low-light problem and to examine the official code structure for training.

3 Paper Summary

3.1 Dataset Overview

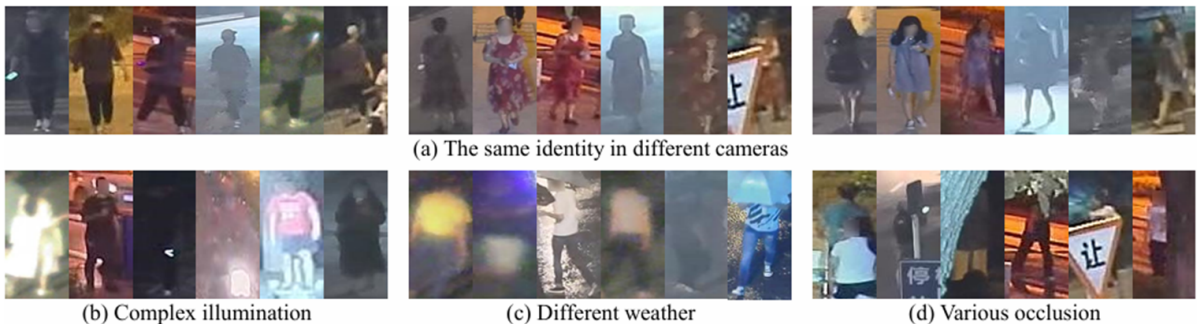


Figure 1: Samples of NightReID dataset

3.2 Proposed Method

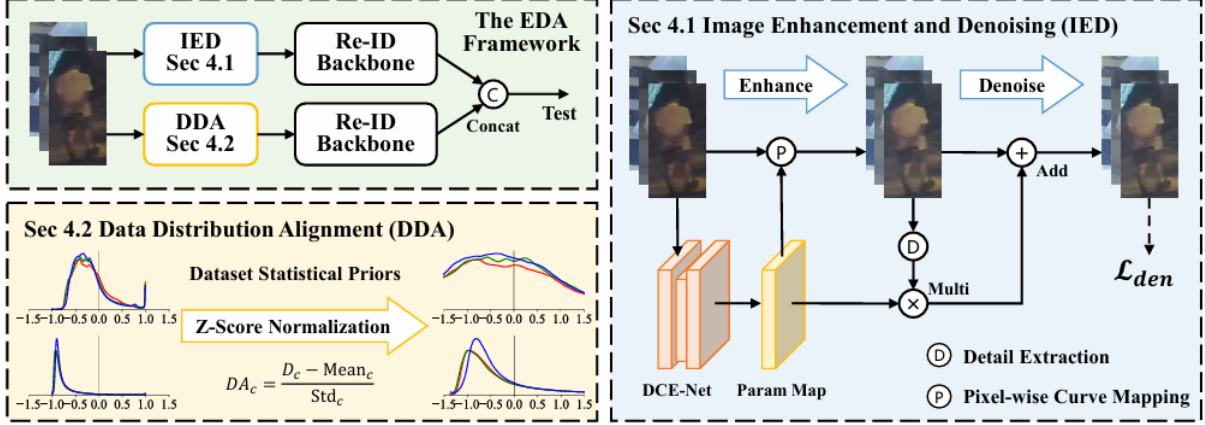


Figure 2: Architecture of EDA

The authors propose an EDA (Enhancement–Denoising–Alignment) framework. The model includes two pre-processing modules:

- **Image Enhancement and Denoising Module:** enhances the visual quality of low-light inputs by jointly correcting illumination and suppressing noise. This module is built upon Zero-DCE [1], which estimates a pixel-wise enhancement curve for effective lightness adjustment.
- **Data Distribution Alignment Module:** mitigates the domain gap between enhanced low-light images and normal-light images by aligning their statistical distributions. Specifically, Z-score normalization is applied to unify per-channel mean and variance before the features are fed into the Re-ID backbone.

After these pre-processing steps, the enhanced images are fed into the Re-ID backbone to extract identity-discriminative features. Next, we shift our attention to the training stage, explaining how the loss functions, optimization scheme, and the overall training loop are designed and executed.

To train the enhancement and denoising modules jointly with the backbone, the authors employ a combination of low-level image reconstruction losses. These losses are adapted from Zero-DCE-based illumination enhancement and edge-preserving denoising formulations. The following code snippet summarizes the implementation of each loss component used in the EDA framework.

4 Code Analysis

4.1 Environment Setup

- Framework: PyTorch 1.8.1
- Hardware: Nvidia A5000
- Dependencies: List key libraries (e.g., ignite, yacs, etc.)

4.2 Key Implementation Details

```

1 ...
2
3 if __name__ == '__main__':
4
5 ...
6
7 train_loader, train_loader_normal, val_loader, num_query, num_classes, camera_num,
  view_num = make_dataloader(cfg)
8

```

```

9 model = make_model(cfg, num_class=num_classes, camera_num=camera_num, view_num=view_num)
10 loss_func, center_criterion = make_loss(cfg, num_classes=num_classes)
11 optimizer, optimizer_center = make_optimizer(cfg, model, center_criterion)
12
13 ...
14
15 do_train(
16     cfg,
17     model,
18     center_criterion,
19     train_loader,
20     val_loader,
21     optimizer,
22     optimizer_center,
23     scheduler,
24     loss_func,
25     num_query, args.local_rank
26 )

```

Listing 1: train.py

As shown in train.py, the training pipeline consists of five components: (1) dataloader, (2) model, (3) loss functions, (4) optimizers, and (5) train loop. To understand the entire system, we first examine the model architecture and then move on to the training procedure.

```

1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4
5
6 class L_color(nn.Module):
7     def __init__(self):
8         super(L_color, self).__init__()
9
10    def forward(self, x ):
11        mean_rgb = torch.mean(x,[2,3],keepdim=True)
12
13        mr,mg, mb = torch.split(mean_rgb, 1, dim=1)
14        Drg = torch.pow(mr-mg,2)
15        Drb = torch.pow(mr-mb,2)
16        Dgb = torch.pow(mb-mg,2)
17
18        k = torch.pow(torch.pow(Drg,2) + torch.pow(Drb,2) + torch.pow(Dgb,2),0.5)
19        return k
20
21
22 class L_spa(nn.Module):
23     def __init__(self):
24         super(L_spa, self).__init__()
25
26        kernel_left = torch.FloatTensor( [[0,0,0],[-1,1,0],[0,0,0]]).cuda().unsqueeze(0)
27        kernel_right = torch.FloatTensor( [[0,0,0],[0,1,-1],[0,0,0]]).cuda().unsqueeze(0)
28        kernel_up = torch.FloatTensor( [[0,-1,0],[0,1, 0],[0,0,0]]).cuda().unsqueeze(0)
29        kernel_down = torch.FloatTensor( [[0,0,0],[0,1, 0],[0,-1,0]]).cuda().unsqueeze(0)
30        self.weight_left = nn.Parameter(data=kernel_left, requires_grad=False)
31        self.weight_right = nn.Parameter(data=kernel_right, requires_grad=False)
32        self.weight_up = nn.Parameter(data=kernel_up, requires_grad=False)
33        self.weight_down = nn.Parameter(data=kernel_down, requires_grad=False)
34        self.pool = nn.AvgPool2d(4)
35
36    def forward(self, org , enhance ):
37        org_mean = torch.mean(org,1,keepdim=True)
38        enhance_mean = torch.mean(enhance,1,keepdim=True)
39
40        org_pool = self.pool(org_mean)
41        enhance_pool = self.pool(enhance_mean)
42
43        D_org_leftf = F.conv2d(org_pool , self.weight_left, padding=1)
44        D_org_right = F.conv2d(org_pool , self.weight_right, padding=1)
45        D_org_up = F.conv2d(org_pool , self.weight_up, padding=1)

```

```

46     D_org_down = F.conv2d(org_pool , self.weight_down, padding=1)
47
48     D_enhance_left = F.conv2d(enhance_pool , self.weight_left, padding=1)
49     D_enhance_right = F.conv2d(enhance_pool , self.weight_right, padding=1)
50     D_enhance_up = F.conv2d(enhance_pool , self.weight_up, padding=1)
51     D_enhance_down = F.conv2d(enhance_pool , self.weight_down, padding=1)
52
53     D_left = torch.pow(D_org_left - D_enhance_left,2)
54     D_right = torch.pow(D_org_right - D_enhance_right,2)
55     D_up = torch.pow(D_org_up - D_enhance_up,2)
56     D_down = torch.pow(D_org_down - D_enhance_down,2)
57     E = (D_left + D_right + D_up +D_down)
58
59     return E
60
61 class L_exp(nn.Module):
62
63     def __init__(self,patch_size,mean_val):
64         super(L_exp, self).__init__()
65         self.pool = nn.AvgPool2d(patch_size)
66         self.mean_val = mean_val
67
68     def forward(self, x):
69         x = torch.mean(x,1,keepdim=True)
70         mean = self.pool(x)
71         d = torch.mean(torch.pow(mean- torch.FloatTensor([self.mean_val] ).cuda(),2))
72         return d
73
74 class L_exp_v2(nn.Module):
75     def __init__(self,patch_size,mean_val):
76         super(L_exp_v2, self).__init__()
77         self.pool = nn.AvgPool2d(patch_size)
78         self.mean_val = torch.FloatTensor([mean_val]).cuda()
79
80     def forward(self, x, x_or):
81         x = torch.mean(x,1,keepdim=True)
82         x_or = torch.mean(x_or,1,keepdim=True)
83         mean = self.pool(x)
84         mean_or = self.pool(x_or)
85         d = torch.mean(torch.pow(mean-(mean_or+self.mean_val)/2,2))
86         return d
87
88 class L_TV(nn.Module):
89     def __init__(self,TVLoss_weight=1):
90         super(L_TV,self).__init__()
91         self.TVLoss_weight = TVLoss_weight
92
93     def forward(self,x):
94         batch_size = x.size()[0]
95         h_x = x.size()[2]
96         w_x = x.size()[3]
97         count_h = (x.size()[2]-1) * x.size()[3]
98         count_w = x.size()[2] * (x.size()[3] - 1)
99         h_tv = torch.pow((x[:, :, 1:, :] - x[:, :, :h_x-1, :]),2).sum()
100         w_tv = torch.pow((x[:, :, :, 1:] - x[:, :, :, :w_x-1]),2).sum()
101         return self.TVLoss_weight*2*(h_tv/count_h+w_tv/count_w)/batch_size
102
103 class L_denoise(nn.Module):
104     def __init__(self):
105         super(L_denoise, self).__init__()
106         self.pool1 = nn.AvgPool2d(kernel_size=5, stride=1, padding=2)
107         self.pool2 = nn.MaxPool2d(kernel_size=7, stride=1, padding=3)
108         self.pool3 = nn.AvgPool2d(kernel_size=7, stride=1, padding=3)
109
110     def forward(self, x, direction):
111         if direction == 'x':
112             grad = x[:, :, :, :-1] - x[:, :, :, 1:]
113         elif direction == 'y':
114             grad = x[:, :, :-1, :] - x[:, :, 1:, :]
115         else:
116             raise ValueError("Direction should be 'x' or 'y'")
117
118         abs_grad = torch.abs(grad)

```

```

119     max_grad = torch.max(torch.max(abs_grad, dim=2, keepdim=True)[0], dim=3, keepdim=True)[0]
120     min_grad = torch.min(torch.min(abs_grad, dim=2, keepdim=True)[0], dim=3, keepdim=True)[0]
121     norm1 = (abs_grad - min_grad) / (max_grad - min_grad + 1e-6)
122
123     with torch.no_grad():
124
125         pool_x=self.pool1(x)
126         if direction == 'x':
127             pool_x = pool_x[:, :, :, :-1] - pool_x[:, :, :, 1:]
128         elif direction == 'y':
129             pool_x = pool_x[:, :, :-1, :] - pool_x[:, :, 1:, :]
130         else:
131             raise ValueError("Direction should be 'x' or 'y'")
132
133         # pool_x=self.pool1(grad)
134
135         pool_grad=torch.abs(pool_x)
136
137         pool_max_grad = self.pool2(pool_grad)
138         pool_min_grad = -self.pool2(-pool_grad)
139
140         pool_max_grad_pool = self.pool3(pool_max_grad)
141         pool_min_grad_pool = self.pool3(pool_min_grad)
142
143         weight = (pool_grad - pool_min_grad_pool) / (pool_max_grad_pool - pool_min_grad_pool + 1e-4)
144
145         loss = norm1 * (weight+1e-2)
146         return loss

```

Listing 2: loss for IED

The EDA framework is trained using several low-level reconstruction losses that guide illumination enhancement and noise suppression:

- **Color Constancy Loss (L_{col}):** This loss corrects color bias commonly observed in nighttime scenes (e.g., yellowish cast from streetlights).
- **Spatial Consistency Loss (L_{spa}):** To preserve the structural information of the original image, L_{spa} minimizes the difference in spatial gradients between the input and the enhanced image.
- **Total Variation Loss (L_{tv}):** Constrains the spatial smoothness of the enhanced image to remove noise while preserving edges.
- **Exposure Control Loss (L_{exp}):** This loss restrains the image from being under- or over-exposed by controlling the average intensity value E to be close to a target level.
- **Denoising Loss (L_{den}):** Since brightening dark images often amplifies noise, we utilize a gradient-based denoising loss. This formulation suppresses noise in uniform regions while preserving meaningful edges.

```

1 class TripletLoss(object):
2
3 def __init__(self, margin=None, hard_factor=0.0):
4     self.margin = margin
5     self.hard_factor = hard_factor
6     if margin is not None:
7         self.ranking_loss = nn.MarginRankingLoss(margin=margin)
8     else:
9         self.ranking_loss = nn.SoftMarginLoss()
10
11 def __call__(self, global_feat, labels, normalize_feature=False):
12     if normalize_feature:
13         global_feat = normalize(global_feat, axis=-1)
14     dist_mat = euclidean_dist(global_feat, global_feat)
15     dist_ap, dist_an = hard_example_mining(dist_mat, labels)
16
17     y = dist_an.new().resize_as_(dist_an).fill_(1)
18     if self.margin is not None:

```

```

19     loss = self.ranking_loss(dist_an, dist_ap, y)
20     else:
21         loss = self.ranking_loss(dist_an - dist_ap, y)
22     return loss, dist_ap, dist_an
23
24 class CrossEntropyLabelSmooth(nn.Module):
25
26     def __init__(self, num_classes, epsilon=0.1, use_gpu=True):
27         super(CrossEntropyLabelSmooth, self).__init__()
28         self.num_classes = num_classes
29         self.epsilon = epsilon
30         self.use_gpu = use_gpu
31         self.logsoftmax = nn.LogSoftmax(dim=1)
32
33     def forward(self, inputs, targets):
34         log_probs = self.logsoftmax(inputs)
35         targets = torch.zeros(log_probs.size()).scatter_(1, targets.unsqueeze(1).data.
36         cpu(), 1)
37         if self.use_gpu: targets = targets.cuda()
38         targets = (1 - self.epsilon) * targets + self.epsilon / self.num_classes
39         loss = (- targets * log_probs).mean(0).sum()
40         return loss

```

Listing 3: loss for Re-ID

For the Re-ID objective, the framework adopts two widely used losses: (1) the ID loss and (2) the Triplet loss. The ID loss is formulated as a standard classification loss with label smoothing, encouraging the model to produce discriminative logits for each identity. The Triplet loss, originally introduced in FaceNet[5], operates on triplets composed of an anchor, a positive sample, and a negative sample; it minimizes the distance between the anchor and positive pair while maximizing the distance between the anchor and negative pair. By jointly using these two losses, the model learns features that are both identity-discriminative and well-separated in the embedding space.

- **Identity Loss (L_{id}):** We use Cross-Entropy loss with label smoothing to prevent overfitting and encourage the model to learn discriminative features for each identity.
- **Triplet Loss (L_{tri}):** This loss ensures that features of the same identity are pulled closer in the embedding space, while features of different identities are pushed apart.

```

1 import logging
2 import os
3 import time
4 import torch
5 import torch.nn as nn
6 from utils.meter import AverageMeter
7 from utils.metrics import R1_mAP_eval
8 from torch.cuda import amp
9 import torch.distributed as dist
10
11 def do_train(cfg,
12             model,
13             center_criterion,
14             train_loader,
15             val_loader,
16             optimizer,
17             optimizer_center,
18             scheduler,
19             loss_fn,
20             num_query, local_rank):
21     log_period = cfg.SOLVER.LOG_PERIOD
22     checkpoint_period = cfg.SOLVER.CHECKPOINT_PERIOD
23     eval_period = cfg.SOLVER.EVAL_PERIOD
24
25     device = "cuda"
26     epochs = cfg.SOLVER.MAX_EPOCHS
27
28     logger = logging.getLogger("transreid.train")
29     logger.info('start training')
30     _LOCAL_PROCESS_GROUP = None
31     if device:
32         model.to(local_rank)

```

```

33         if torch.cuda.device_count() > 1 and cfg.MODEL.DIST_TRAIN:
34             logger.info('Using {} GPUs for training'.format(torch.cuda.device_count()))
35             model = torch.nn.parallel.DistributedDataParallel(model, device_ids=[
local_rank], find_unused_parameters=True)
36
37     loss_meter = AverageMeter()
38     acc_meter = AverageMeter()
39
40     evaluator = R1_mAP_eval(num_query, max_rank=50, feat_norm=cfg.TEST.FEAT_NORM)
41     scaler = amp.GradScaler()
42     best_mAP = 0
43     # train
44     for epoch in range(1, epochs + 1):
45         start_time = time.time()
46         loss_meter.reset()
47         acc_meter.reset()
48         evaluator.reset()
49         model.train()
50         for n_iter, (img, vid, target_cam, target_view) in enumerate(train_loader):
51             optimizer.zero_grad()
52             optimizer_center.zero_grad()
53             img = img.to(device)
54             target = vid.to(device)
55             target_cam = target_cam.to(device)
56             target_view = target_view.to(device)
57             with amp.autocast(enabled=True):
58                 loss=0
59                 score_i, feat_i, score_d, feat_d, score_c, feat_c, loss_dce = model(img,
target, cam_label=target_cam, view_label=target_view)
60                 loss += loss_fn(score_i, feat_i, target, target_cam)
61                 loss += loss_fn(score_d, feat_d, target, target_cam)
62                 loss += loss_fn(score_c, feat_c, target, target_cam)
63                 loss += loss_dce*cfg.MODEL.ENHANCE_WEIGHT
64             scaler.scale(loss).backward()
65
66             scaler.step(optimizer)
67             scaler.update()
68
69             if 'center' in cfg.MODEL.METRIC_LOSS_TYPE:
70                 for param in center_criterion.parameters():
71                     param.grad.data *= (1. / cfg.SOLVER.CENTER_LOSS_WEIGHT)
72                 scaler.step(optimizer_center)
73                 scaler.update()
74             if isinstance(score_c, list):
75                 acc = (score_c[0].max(1)[1] == target).float().mean()
76             else:
77                 acc = (score_c.max(1)[1] == target).float().mean()
78
79             loss_meter.update(loss.item(), img.shape[0])
80             acc_meter.update(acc, 1)
81
82         torch.cuda.synchronize()
83         if cfg.MODEL.DIST_TRAIN:
84             if dist.get_rank() == 0:
85                 if (n_iter + 1) % log_period == 0:
86                     base_lr = scheduler._get_lr(epoch)[0] if cfg.SOLVER.
WARMUP_METHOD == 'cosine' else scheduler.get_lr()[0]
87                     logger.info("Epoch[{}] Iter[{} / {}] Loss: {:.3f}, Acc: {:.3f},
Base Lr: {:.2e}"
88                                 .format(epoch, (n_iter + 1), len(train_loader),
loss_meter.avg, acc_meter.avg, base_lr))
89                 else:
90                     if (n_iter + 1) % log_period == 0:
91                         base_lr = scheduler._get_lr(epoch)[0] if cfg.SOLVER.WARMUP_METHOD ==
'cosine' else scheduler.get_lr()[0]
92                         logger.info("Epoch[{}] Iter[{} / {}] Loss: {:.3f}, Acc: {:.3f}, Base
Lr: {:.2e}"
93                                     .format(epoch, (n_iter + 1), len(train_loader),
loss_meter.avg, acc_meter.avg, base_lr))
94
95         end_time = time.time()
96         time_per_batch = (end_time - start_time) / (n_iter + 1)
97         if cfg.SOLVER.WARMUP_METHOD == 'cosine':

```



```

98         scheduler.step(epoch)
99     else:
100         scheduler.step()
101     if cfg.MODEL.DIST_TRAIN:
102         pass
103     else:
104         logger.info("Epoch {} done. Time per epoch: {:.3f}[s] Speed: {:.1f}[samples/
s]"
105                     .format(epoch, time_per_batch * (n_iter + 1), train_loader.
batch_size / time_per_batch))
106
107     if epoch % checkpoint_period == 0:
108         if cfg.MODEL.DIST_TRAIN:
109             if dist.get_rank() == 0:
110                 torch.save(model.state_dict(),
111                             os.path.join(cfg.OUTPUT_DIR, cfg.MODEL.NAME + '_{}.pth'.
format(epoch)))
112             else:
113                 torch.save(model.state_dict(),
114                             os.path.join(cfg.OUTPUT_DIR, cfg.MODEL.NAME + '_{}.pth'.
format(epoch)))
115
116     if epoch % eval_period == 0:
117         if cfg.MODEL.DIST_TRAIN:
118             if dist.get_rank() == 0:
119                 model.eval()
120                 for n_iter, (img, vid, camid, camids, target_view, _) in enumerate(
val_loader):
121                     with torch.no_grad():
122                         img = img.to(device)
123                         camids = camids.to(device)
124                         target_view = target_view.to(device)
125                         feat_i, feat_d, feat_c = model(img, cam_label=camids,
view_label=target_view)
126                         evaluator.update((feat_i, feat_d, feat_c, vid, camid))
127                     cmc_ls, mAP_ls, _, _, _, _ = evaluator.compute()
128                     logger.info("Validation Results - Epoch: {}".format(epoch))
129                     for i in range(len(cmc_ls)):
130                         logger.info("mode: {}".format(i))
131                         logger.info("mAP: {:.1%}".format(mAP_ls[i]))
132                         for r in [1, 5, 10]:
133                             logger.info("CMC curve, Rank-{:<3}:{:.1%}".format(r, cmc_ls[
i][r - 1]))
134                     if mAP_ls[-1] > best_mAP:
135                         best_mAP = mAP_ls[-1]
136                         torch.save(model.state_dict(), os.path.join(cfg.OUTPUT_DIR, cfg.
MODEL.NAME + '_best_mAP.pth'))
137                         torch.cuda.empty_cache()
138                 else:
139                     model.eval()
140                     for n_iter, (img, vid, camid, camids, target_view, _) in enumerate(
val_loader):
141                         with torch.no_grad():
142                             img = img.to(device)
143                             camids = camids.to(device)
144                             target_view = target_view.to(device)
145                             feat_i, feat_d, feat_c = model(img, cam_label=camids, view_label
=target_view)
146                             evaluator.update((feat_i, feat_d, feat_c, vid, camid))
147                         cmc_ls, mAP_ls, _, _, _, _ = evaluator.compute()
148                         logger.info("Validation Results - Epoch: {}".format(epoch))
149                         for i in range(len(cmc_ls)):
150                             logger.info("mode: {}".format(i))
151                             logger.info("mAP: {:.1%}".format(mAP_ls[i]))
152                             for r in [1, 5, 10]:
153                                 logger.info("CMC curve, Rank-{:<3}:{:.1%}".format(r, cmc_ls[i][r
- 1]))
154                         if mAP_ls[-1] > best_mAP:
155                             best_mAP = mAP_ls[-1]
156                             torch.save(model.state_dict(), os.path.join(cfg.OUTPUT_DIR, cfg.
MODEL.NAME + '_best_mAP.pth'))
157                             torch.cuda.empty_cache()

```

Listing 4: processor.py

The overall training procedure is implemented in the `do_train` function of `processor.py`. It handles multi-GPU training, logging, evaluation, and checkpointing in an end-to-end manner.

At the beginning, the model is moved to the target GPU (`local_rank`), and if distributed training is enabled (`cfg.MODEL.DIST_TRAIN`), the model is wrapped by `DistributedDataParallel` to synchronize gradients across multiple GPUs. Several meters such as `loss_meter` and `acc_meter` are initialized to record the training statistics, and an `R1_mAP_eval` evaluator is prepared to compute Rank-1 accuracy and mAP on the validation set. Mixed-precision training is enabled via `torch.cuda.amp.GradScaler`, which stabilizes training while reducing memory usage.

For each epoch, the function resets the meters and sets the model to training mode. Then, it iterates over the mini-batches from `train_loader`. Each batch contains an input image tensor and corresponding labels: identity (`vid`), camera ID (`target_cam`), and view ID (`target_view`). These tensors are transferred to GPU, and a forward pass is executed within an `amp.autocast` context.

The model outputs three branches of classification scores and features (`(score_i, feat_i)`, `(score_d, feat_d)`, `(score_c, feat_c)`) corresponding to different branches (e.g., image, denoised, and combined features), along with the enhancement loss term `loss_dce` from the EDA module. The total Re-ID loss is computed by applying the same loss function `loss_fn` (ID loss + Triplet loss) to each branch and summing them up:

$$L_{\text{ReID}} = L(\text{score}_i, \text{feat}_i) + L(\text{score}_d, \text{feat}_d) + L(\text{score}_c, \text{feat}_c).$$

In addition, the enhancement loss is weighted by `cfg.MODEL.ENHANCE_WEIGHT` and added to the total loss:

$$L_{\text{total}} = L_{\text{ReID}} + \lambda_{\text{enh}} \cdot L_{\text{DCE}}.$$

This total loss is backpropagated using `GradScaler`, and the main optimizer is updated accordingly. If center loss is enabled, the gradient of the center parameters is rescaled by the inverse of `CENTER_LOSS_WEIGHT` and the center optimizer is also stepped.

During training, classification accuracy is computed from `score_c` and recorded by `acc_meter`, while `loss_meter` tracks the running average of the training loss. At a fixed logging interval (`LOG_PERIOD`), the current epoch, iteration, averaged loss, accuracy, and base learning rate are written to the logger.

After each epoch, the learning rate scheduler is updated (either cosine warmup or step scheduler), and the epoch-level training time and throughput (samples per second) are reported. The model parameters are periodically saved to disk according to `CHECKPOINT_PERIOD`.

For evaluation, the model is switched to evaluation mode and the validation loader is iterated without gradient computation. For each batch, the three feature branches (`feat_i`, `feat_d`, `feat_c`) are extracted and accumulated in the evaluator. After all validation samples are processed, the evaluator computes CMC and mAP for each mode. If the mAP of the last mode (typically the combined feature) surpasses the best mAP observed so far, the current model is stored as the best checkpoint. This procedure allows the framework to jointly optimize low-level enhancement, denoising, and high-level Re-ID feature learning in a unified training loop.

The training procedure, implemented in `do_train`, optimizes both the enhancement module and the Re-ID backbone in an end-to-end manner. The overall pipeline is as follows:

1. **Forward Pass:** A batch of low-light images is fed into the EDA framework. The model outputs the enhanced images, denoised features, and the final identity features.
2. **Joint Loss Calculation:** The total loss L_{total} is a weighted sum of the Re-ID loss and the enhancement loss. It is formulated as:

$$L_{\text{total}} = L_{\text{ReID}} + \lambda_{\text{enh}} \cdot L_{\text{DCE}} \quad (1)$$

where $L_{\text{ReID}} = L_{\text{id}} + L_{\text{tri}}$ and L_{DCE} represents the sum of enhancement losses (L_{col} , L_{spa} , L_{exp} , L_{den}).

3. **Optimization:** We use *Automatic Mixed Precision (AMP)* to reduce memory consumption and accelerate training. The gradients are computed based on L_{total} , updating both the low-level enhancement layers and high-level feature extraction layers simultaneously.

5 Experiment and Results

5.1 Evaluation Metrics

To evaluate the performance of the NightReID model, we utilize standard person re-identification metrics: Cumulative Match Characteristic (CMC) and mean Average Precision (mAP).

- **Rank-1 Accuracy (CMC):** This metric measures the probability that the correct match appears in the top-1 retrieved result. It indicates the model’s ability to correctly identify the target person immediately.
- **mean Average Precision (mAP):** mAP considers both precision and recall, providing a comprehensive measure of retrieval performance. It is particularly important in surveillance scenarios where finding all instances of a target person is crucial.

5.2 Experiment Results

Table 1: Performance of EDA on NightReID (528 IDs w/ Distractor)

Method	mAP(%)	R-1(%)	R-5(%)	R-10(%)
EDA (paper)	52.3	67.6	-	-
EDA (experiment)	51.4	66.6	78.0	84.7

A distractor refers to an ID that does not appear in the query set. Because real surveillance environments often contain many irrelevant individuals, including distractors makes the evaluation setting more realistic. The NightReID dataset provides four test configurations: two ID scales (528 IDs and 1000 IDs), each with and without distractors. Among them, the 528 IDs w/ Distractor (528IDs w/D) setting is considered the default evaluation protocol, and we therefore adopt this setting in our experiments.

The results from our reproduction closely matched those reported in the original paper. The paper reports 52.3% mAP and 67.6% Rank-1, while our experiment achieved 51.4% mAP and 66.6% Rank-1, showing only a 0.9 percentage point difference in mAP. The paper does not report Rank-5 or Rank-10 performance, but our reproduction achieved 78.0% Rank-5 and 84.7% Rank-10, respectively.

References

- [1] Chunle Guo Guo, Chongyi Li, Jichang Guo, Chen Change Loy, Junhui Hou, Sam Kwong, and Runmin Cong. Zero-reference deep curve estimation for low-light image enhancement. In *Proceedings of the IEEE conference on computer vision and pattern recognition (CVPR)*, pages 1780–1789, June 2020.
- [2] Shuting He, Hao Luo, Pichao Wang, Fan Wang, Hao Li, and Wei Jiang. Transreid: Transformer-based object re-identification, 2021.
- [3] Hao Luo, Youzhi Gu, Xingyu Liao, Shenqi Lai, and Wei Jiang. Bag of tricks and a strong baseline for deep person re-identification, 2019.
- [4] Hao Luo, Pichao Wang, Yi Xu, Feng Ding, Yanxin Zhou, Fan Wang, Hao Li, and Rong Jin. Self-supervised pre-training for transformer-based person re-identification, 2021.
- [5] Florian Schroff, Dmitry Kalenichenko, and James Philbin. Facenet: A unified embedding for face recognition and clustering. In *2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, page 815–823. IEEE, June 2015.